RBE577: Machine Learning for Robotics Project 3: Act or Critic: Learning to Pick

Filippo Marcantoni
Robotics Engineering, WPI
fmarcantoni@wpi.edu

Prahladh Raja
Robotics Engineering, WPI
pnamboorkrishnam@wpi.edu

***Abstract*—This report investigates three reinforcement learning formulations spanning tabular decision-making, policy-gradient control, and vision-based robotic manipulation. First, a finite-state Markov Decision Process is analyzed using policy iteration to illustrate exact dynamic programming over discrete states and actions. Second, the policy gradient theorem is derived and used to implement REINFORCE and Advantage Actor-Critic (A2C) for the `LunarLander-v2` benchmark, highlighting the role of value-based baselines in reducing variance and improving learning stability. Third, an Asynchronous Advantage Actor-Critic (A3C) framework is developed for a robotic grasping task in PyBullet, where the agent must learn a continuous control policy directly from visual observations. Together, these experiments examine how reinforcement learning methods scale from analytically tractable problems to high-dimensional control tasks, and how architectural and algorithmic choices affect policy quality, training stability, and sample efficiency.**

I. Policy Iteration

A. Problem Setup

Consider the Markov Decision Process with state space $\mathcal{S} = \{A, B, C\}$, action space $\mathcal{A} = \{X, Y\}$, and discount factor $\gamma = 0.9$. The immediate reward matrix is

$$R = \begin{bmatrix} 5 & -2 \\ 1 & 2 \\ -1 & 0 \end{bmatrix},$$

where $R[i, j]$ denotes the reward obtained by taking action j in state i. The transition matrices for actions X and Y are

$$P_X = \begin{bmatrix} 0.3 & 0.6 & 0.1 \\ 0.7 & 0.2 & 0.1 \\ 0.4 & 0.4 & 0.2 \end{bmatrix}, \qquad P_Y = \begin{bmatrix} 0.2 & 0.4 & 0.4 \\ 0.1 & 0.6 & 0.3 \\ 0.3 & 0.3 & 0.4 \end{bmatrix}.$$

The initial policy is chosen as

$$\pi^{(0)}(A) = \pi^{(0)}(B) = \pi^{(0)}(C) = X.$$

B. Iteration 1: Policy Evaluation

For the initial policy, the Bellman equations are

$$\begin{aligned} V(A) &= 5 + 0.9\,[0.3V(A) + 0.6V(B) + 0.1V(C)], \\ V(B) &= 1 + 0.9\,[0.7V(A) + 0.2V(B) + 0.1V(C)], \\ V(C) &= -1 + 0.9\,[0.4V(A) + 0.4V(B) + 0.2V(C)]. \end{aligned} \quad (1)$$

Rearranging terms gives

$$\begin{aligned} 0.73V(A) - 0.54V(B) - 0.09V(C) &= 5, \\ -0.63V(A) + 0.82V(B) - 0.09V(C) &= 1, \\ -0.36V(A) - 0.36V(B) + 0.82V(C) &= -1. \end{aligned} \quad (2)$$

Solving this linear system yields

$$\boxed{V^{\pi^{(0)}}(A) \approx 28.70, \quad V^{\pi^{(0)}}(B) \approx 25.76, \quad V^{\pi^{(0)}}(C) \approx 22.70.}$$

C. Iteration 1: Policy Improvement

The state-action values are computed as

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P_a(s, s') V^{\pi^{(0)}}(s').$$

a) State A:

$$\begin{aligned} Q(A, X) &= 5 + 0.9\,[0.3(28.70) + 0.6(25.76) + 0.1(22.70)] = 28.70, \\ Q(A, Y) &= -2 + 0.9\,[0.2(28.70) + 0.4(25.76) + 0.4(22.70)] = 20.61. \end{aligned}$$

Hence, $\pi^{(1)}(A) = X$.

b) State B:

$$\begin{aligned} Q(B, X) &= 1 + 0.9\,[0.7(28.70) + 0.2(25.76) + 0.1(22.70)] = 25.76, \\ Q(B, Y) &= 2 + 0.9\,[0.1(28.70) + 0.6(25.76) + 0.3(22.70)] = 24.62. \end{aligned}$$

Hence, $\pi^{(1)}(B) = X$.

c) State C:

$$\begin{aligned} Q(C, X) &= -1 + 0.9\,[0.4(28.70) + 0.4(25.76) + 0.2(22.70)] = 22.70, \\ Q(C, Y) &= 0 + 0.9\,[0.3(28.70) + 0.3(25.76) + 0.4(22.70)] = 22.88. \end{aligned}$$

Hence, $\pi^{(1)}(C) = Y$.

Therefore, the improved policy after the first iteration is

$$\pi^{(1)}(A) = X, \qquad \pi^{(1)}(B) = X, \qquad \pi^{(1)}(C) = Y.$$

D. Iteration 2: Policy Evaluation

Under $\pi^{(1)}$, states A and B continue to use action X, whereas state C now uses action Y. The Bellman equations become

$$\begin{aligned} V(A) &= 5 + 0.9\,[0.3V(A) + 0.6V(B) + 0.1V(C)], \\ V(B) &= 1 + 0.9\,[0.7V(A) + 0.2V(B) + 0.1V(C)], \\ V(C) &= 0 + 0.9\,[0.3V(A) + 0.3V(B) + 0.4V(C)]. \end{aligned} \quad (3)$$

Solving this updated system gives

$$\boxed{V^{\pi^{(1)}}(A) \approx 28.92, \quad V^{\pi^{(1)}}(B) \approx 25.99, \quad V^{\pi^{(1)}}(C) \approx 23.17.}$$

E. Iteration 2: Policy Improvement

Using $V^{\pi^{(1)}}$, the updated state-action values are

$$Q(A,X) = 28.95, \qquad Q(A,Y) = 20.92,$$
$$Q(B,X) = 26.01, \qquad Q(B,Y) = 24.91,$$
$$Q(C,X) = 22.96, \qquad Q(C,Y) = 23.19.$$

The greedy choices remain unchanged:

$$\pi^{(2)}(A) = X, \qquad \pi^{(2)}(B) = X, \qquad \pi^{(2)}(C) = Y.$$

Since $\pi^{(2)} = \pi^{(1)}$, policy iteration has converged. The optimal policy is therefore

$$\boxed{\pi^*(A) = X, \qquad \pi^*(B) = X, \qquad \pi^*(C) = Y.}$$

II. Deep Reinforcement Learning

A. Policy Gradient Derivation

Let $\pi_\theta(a|s)$ denote a parameterized stochastic policy and let $\tau = (s_0, a_0, \ldots, s_T)$ denote a trajectory sampled from the environment. The trajectory distribution is

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t)\, p(s_{t+1}|s_t, a_t). \tag{4}$$

The reinforcement learning objective is to maximize the expected return

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[R(\tau)] = \int p_\theta(\tau) R(\tau)\, d\tau, \tag{5}$$

where

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r(s_t, a_t). \tag{6}$$

Taking the gradient of the objective yields

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau)\, R(\tau)\, d\tau. \tag{7}$$

Applying the log-derivative identity $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$ gives

$$\nabla_\theta J(\theta) = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, d\tau = \mathbb{E}_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau)\, R(\tau) \tag{8}$$

Expanding the log-probability of the trajectory,

$$\log p_\theta(\tau) = \log p(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t|s_t) + \sum_{t=0}^{T-1} \log p(s_{t+1}|s_t, a_t). \tag{9}$$

Since the environment dynamics and initial-state distribution do not depend on θ,

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t). \tag{10}$$

Substituting this expression into the objective gradient yields the policy gradient theorem:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right) R(\tau) \right].} \tag{11}$$

Using the causality assumption, the full return can be replaced by the reward-to-go

$$G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{t'}, a_{t'}), \tag{12}$$

which gives the Monte Carlo REINFORCE estimator

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta(a_t^i|s_t^i)\, G_t^i. \tag{13}$$

B. REINFORCE on `LunarLander-v2`

1) Environment: The `LunarLander-v2` environment models a two-dimensional spacecraft that must land safely on a designated platform. The state is an 8-dimensional continuous vector

$$s = (x, y, \dot{x}, \dot{y}, \theta, \dot{\theta}, c_L, c_R),$$

where (x, y) represent position, $(\dot{x}, \dot{y})$ denote linear velocity, θ and $\dot{\theta}$ denote orientation and angular velocity, and $c_L, c_R \in \{0, 1\}$ indicate whether the left and right landing legs are in contact with the ground. The action space is discrete with four actions: do nothing, fire the left orientation engine, fire the main engine, and fire the right orientation engine.

The reward function encourages proximity to the landing pad, low velocity, upright attitude, and fuel efficiency. Successful landing yields a reward of approximately $+100$, whereas crashing incurs a penalty of approximately -100. Episodes terminate upon successful landing, crash, or reaching the maximum horizon.

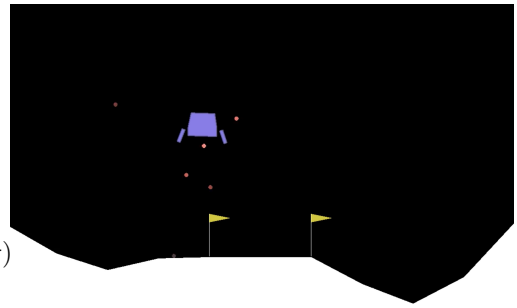

Fig. 1. The `LunarLander-v2` environment. The agent selects discrete thruster actions to achieve a controlled landing on the pad.

2) Implementation: The actor is implemented as a multilayer perceptron with architecture

$$8 \to 256 \to 256 \to 4,$$

with ReLU activations in the hidden layers and categorical action logits at the output. During training, complete episodes are sampled, discounted returns are computed by reverse accumulation, and the returns are normalized to zero mean and unit variance. The objective minimized during training is

$$\mathcal{L}_{\text{actor}} = -\frac{1}{T} \sum_{t=0}^{T-1} \log \pi_\theta(a_t|s_t)\, \hat{G}_t, \tag{14}$$

where $\hat{G}_t$ denotes the normalized return.

To improve numerical stability, the observations are normalized online using Welford's running mean and variance. Gradient clipping is also applied to prevent excessively large updates after high-variance trajectories.

3) Hyperparameters and Training: The REINFORCE agent was trained for 5000 episodes using a stochastic policy optimized with Monte Carlo policy gradients. Hyperparameters were selected to balance training stability and sufficient exploration in the `LunarLander-v2` environment. In particular, observation normalization and gradient clipping were included to reduce instability caused by high-variance return estimates.

TABLE I
REINFORCE HYPERPARAMETERS.

Parameter	Value
Hidden dimension	256
Learning rate	3×10^{-4}
Discount factor γ	0.99
Gradient clipping (max norm)	0.5
Training episodes	5000
Observation normalization	Welford running mean/variance

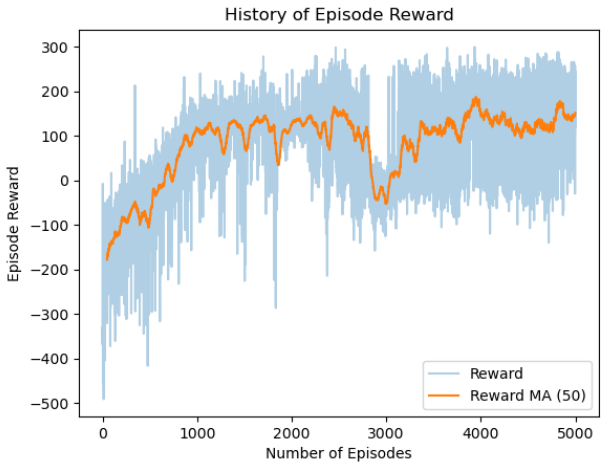

Fig. 2. REINFORCE training reward over 5000 episodes. The light curve denotes per-episode reward, and the bold curve denotes the 100-episode moving average.

4) Evaluation: The best checkpoint is evaluated deterministically over 500 episodes using greedy action selection. The resulting performance is

$$\text{mean reward} = 144.0, \qquad \text{standard deviation} = 60.8.$$

This satisfies the assignment requirement of a mean reward above 100. The relatively high standard deviation reflects the known sensitivity of Monte Carlo policy gradient methods to return variance when no learned baseline is used.

Three evaluation videos are included for qualitative inspection: `reinforce_video_1.mp4`, `reinforce_video_2.mp4`, and `reinforce_video_3.mp4`, with episode rewards of 43.5, 186.8, and 102.4, respectively.

C. Advantage Actor-Critic (A2C)

1) Implementation: To reduce the variance of the policy gradient, the return is replaced by the advantage function

$$A_t = G_t - V_\phi(s_t), \tag{15}$$

where $V_\phi(s_t)$ is a learned baseline produced by a critic network. The actor uses the same architecture as in REINFORCE, while the critic uses a parallel multilayer perceptron with architecture

$$8 \to 256 \to 256 \to 1.$$

Advantages are normalized within each episode. The combined optimization objective is

$$\mathcal{L} = \underbrace{-\frac{1}{T}\sum_t \log \pi_\theta(a_t|s_t)\hat{A}_t}_{\mathcal{L}_{\text{actor}}} + 0.5 \underbrace{\frac{1}{T}\sum_t (G_t - V_\phi(s_t))^2}_{\mathcal{L}_{\text{critic}}}, \tag{16}$$

where $\hat{A}_t$ denotes the normalized advantage. Separate optimizers are used for the actor and critic, and gradient clipping is applied independently to both networks.

2) Hyperparameters and Training: The A2C agent was trained for 5000 episodes using separate actor and critic networks optimized jointly. The hyperparameters were chosen to ensure stable critic learning while allowing the policy to improve gradually through advantage-based updates. As in the REINFORCE implementation, observation normalization and gradient clipping were used to improve optimization robustness.

TABLE II
A2C HYPERPARAMETERS.

Parameter	Value
Hidden dimension	256
Actor learning rate	1×10^{-4}
Critic learning rate	3×10^{-4}
Value loss coefficient	0.5
Discount factor γ	0.99
Gradient clipping (max norm)	0.5
Training episodes	5000
Observation normalization	Welford running mean/variance

3) Evaluation: Deterministic evaluation over 500 episodes yields

$$\text{mean reward} = 241.3, \qquad \text{standard deviation} = 41.4.$$

This substantially exceeds the assignment requirement and indicates a more stable and effective policy than REINFORCE. Three evaluation videos are provided: `a2c_video_1.mp4`, `a2c_video_2.mp4`, and `a2c_video_3.mp4`, with episode rewards of 253.5, 243.0, and 262.6, respectively.

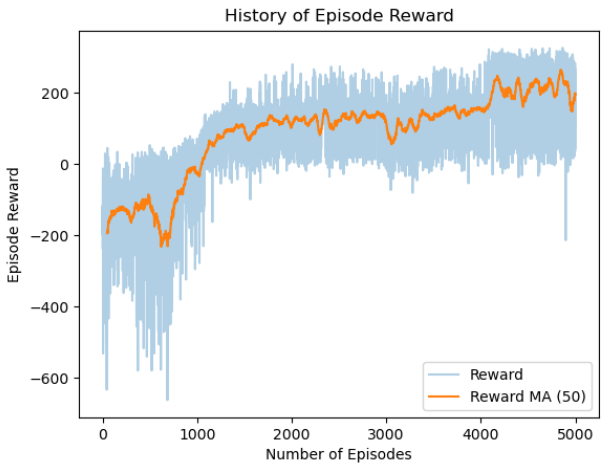

Fig. 3. A2C training reward over 5000 episodes. Relative to REINFORCE, the value baseline reduces variance and improves convergence speed.

D. Comparison of REINFORCE and A2C

TABLE III
COMPARISON OF REINFORCE AND A2C OVER 500 DETERMINISTIC EVALUATION EPISODES.

Metric	REINFORCE	A2C
Mean reward	144.0	**241.3**
Standard deviation	60.8	**41.4**
Passes threshold (≥ 100)	Yes	Yes

A2C improves the mean evaluation reward by 67.6% relative to REINFORCE and reduces reward variance by 31.9%. This improvement is consistent with theory: REINFORCE weights each policy update by the full return, which has high variance, whereas A2C subtracts a learned baseline and updates the actor according to relative advantage. As a result, A2C provides both faster convergence and more reliable final performance, at the cost of learning an additional critic network.

E. A3C for Vision-Based Robotic Picking

1) Environment: The robotic grasping task uses PyBullet's `KukaDiverseObjectEnv`, in which a 7-DOF Kuka IIWA arm must grasp randomly placed objects from a bin. Observations are RGB images captured from an overhead camera. Raw images are normalized to $[0, 1]$, converted to channel-first format, and resized to $3 \times 40 \times 40$ before being passed to the neural network.

With `removeHeightHack=false`, the end-effector descends automatically at a fixed rate, leaving the policy to control horizontal displacement and wrist rotation through a 3-dimensional continuous action vector

$$(dx, dy, d\alpha).$$

Episodes are capped at 20 steps. Once the end-effector falls below a fixed height threshold, the simulator automatically executes a grasp-and-lift sequence. Rewards are binary: 1 for a successful grasp and 0 otherwise.

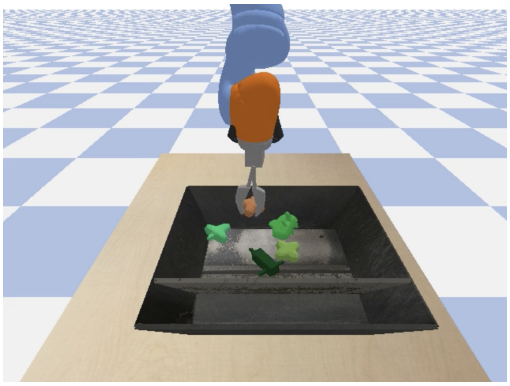

Fig. 4. PyBullet Kuka grasping environment. The robot must infer grasp-relevant visual structure from pixels and output continuous motor commands.

2) A3C Architecture Overview: Asynchronous Advantage Actor-Critic (A3C) trains multiple workers in parallel, each interacting with its own environment instance and maintaining a local copy of the actor-critic network. After collecting a short rollout, each worker computes local returns, advantages, and gradients, then applies those gradients to a shared global model. The local network is subsequently synchronized with the updated shared weights.

Relative to the earlier actor-critic implementation, A3C differs in two important respects. First, experience is gathered asynchronously from multiple environments, which reduces temporal correlation in the training data. Second, updates are performed after short rollouts rather than full episodes, using bootstrapped value estimates when necessary.

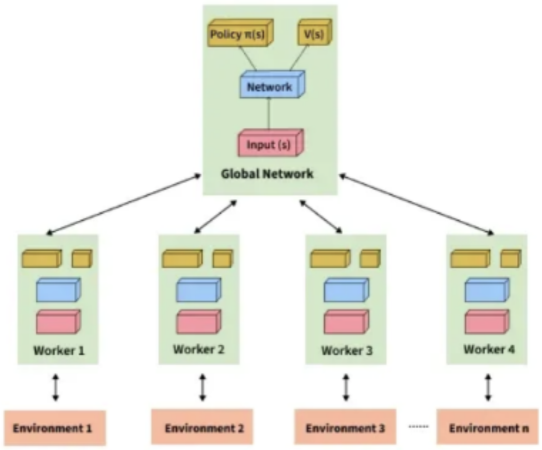

Fig. 5. A3C architecture. Multiple asynchronous workers interact with separate environments and update a shared global actor-critic network.

3) CNN Actor-Critic Network: The network consists of a shared convolutional encoder followed by separate actor and critic heads.

a) Shared encoder:

$$\text{Conv}(3 \to 32,\ 3 \times 3,\ \text{stride } 2,\ \text{pad } 1) \to \text{ReLU}$$

$\rightarrow$ Conv$(32\rightarrow64,\ 3\times3,\ \text{stride } 2,\ \text{pad } 1)$ $\rightarrow$ ReLU

$\rightarrow$ Conv$(64\rightarrow64,\ 3\times3,\ \text{stride } 2,\ \text{pad } 1)$ $\rightarrow$ ReLU

$\rightarrow$ AdaptiveAvgPool(2×2) $\rightarrow$ Flatten(256) $\rightarrow$ FC$(256, 128)$ $\rightarrow$ ReLU $\rightarrow$ FC$(128, 64)$ $\rightarrow$ ReLU.

b) Critic head:

$$\text{FC}(64, 64) \rightarrow \text{ReLU} \rightarrow \text{FC}(64, 1).$$

c) Actor head:

$$\text{FC}(64, 64) \rightarrow \text{ReLU} \rightarrow \text{FC}(64, 3),$$

which outputs the Gaussian mean μ. A learnable standard deviation vector is parameterized using a softplus transform. Actions are sampled according to

$$a \sim \tanh\big(\mathcal{N}\big(\mu, \text{diag}(\sigma^2)\big)\big)\,, \tag{17}$$

which bounds the action values to $[-1, 1]$. Xavier uniform initialization is used throughout the network.

4) Objectives: Each worker collects rollouts of length $t_{\max}$ and computes bootstrapped returns

$$G_t = r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k}), \tag{18}$$

where the bootstrap term is included only when the rollout terminates before the episode ends. The corresponding advantage is

$$A_t = G_t - V(s_t). \tag{19}$$

The actor loss is

$$\mathcal{L}_{\text{actor}} = -\frac{1}{k}\sum_t \log \pi_\theta(a_t|s_t) A_t - \beta \frac{1}{k}\sum_t \mathcal{H}[\pi_\theta(\cdot|s_t)]\,, \tag{20}$$

where $\beta = 0.01$ is the entropy coefficient. Entropy is computed from the pre-`tanh` Gaussian distribution. The critic loss is

$$\mathcal{L}_{\text{critic}} = \frac{1}{k}\sum_t \big(G_t - V(s_t)\big)^2\,. \tag{21}$$

The full objective is

$$\mathcal{L} = \mathcal{L}_{\text{actor}} + 0.5\,\mathcal{L}_{\text{critic}}. \tag{22}$$

Local gradients are applied to the shared global model using `SharedAdam`, whose optimizer state is stored in shared memory. After each update, worker parameters are synchronized with the global network.

5) Hyperparameters and Training: Training was performed on the WPI Turing HPC cluster using two CPU and two GPU workers in headless mode. The `spawn` multiprocessing start method was used to ensure compatibility with PyTorch shared-memory tensors. The full training run of 10,000 episodes completed in approximately three hours.

6) Evaluation: The final checkpoint is evaluated deterministically over 100 episodes by using the mean action $\tanh(\mu)$ without sampling.

The learned policy achieves a 55% grasp success rate, more than double the required 25% threshold. The average episode length of seven steps suggests that the policy quickly converges to a grasp attempt rather than wandering in the action space. An evaluation video containing successful and failed grasp attempts is provided as `a3c_kuka_video.mp4`.

TABLE IV
A3C HYPERPARAMETERS.

Parameter	Value
Number of workers	4
Rollout length $t_{\max}$	20
Discount factor γ	0.99
Learning rate	2×10^{-4}
Entropy coefficient β	0.01
Value loss coefficient	0.5
Gradient clipping (max norm)	40.0
Total episodes	10,000
CNN input size	$3 \times 40 \times 40$
Optimizer	SharedAdam
Weight initialization	Xavier uniform

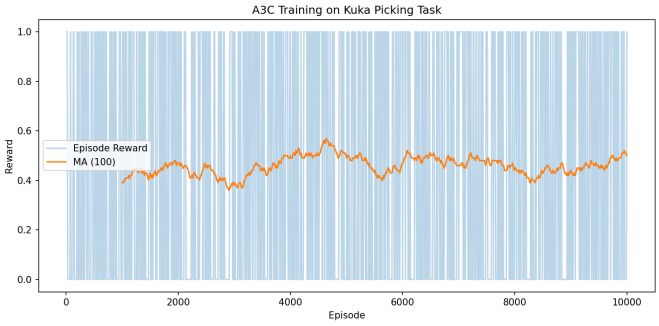

Fig. 6. A3C training curve over 10,000 episodes. Because the reward is binary, the 100-episode moving average directly represents the grasp success rate.

TABLE V
A3C EVALUATION OVER 100 EPISODES.

Metric	Value
Successes	55 / 100
Success rate	**55.0%**
Average reward	0.550 ± 0.498
Average episode length	7.0 steps

III. DISCUSSION

A. Effect of the Advantage Baseline

The clearest trend in the LunarLander experiments is the benefit of replacing raw returns with an advantage estimate. REINFORCE updates are scaled directly by G_t, which can fluctuate substantially from episode to episode. A2C reduces this variance by subtracting a learned value baseline. Empirically, this leads to both higher final reward and lower variability: the mean reward increases from 144.0 to 241.3, while the standard deviation decreases from 60.8 to 41.4. The training curves also show that A2C converges faster and more smoothly.

B. Stabilization Techniques

Training stability was strongly influenced by feature normalization and gradient control. In LunarLander, the eight state variables span different numerical ranges, so online observation normalization prevents any single component from dom-

inating the gradient signal. Gradient clipping further protects training from rare but destructive parameter updates. Similar stabilization was necessary for A3C, where asynchronous updates and sparse rewards can otherwise amplify optimization noise.

C. Why A3C Is Effective for Vision-Based Control

The robotic grasping task is substantially more difficult than LunarLander because the agent must jointly learn perception and control from image observations. A3C is well suited to this setting for three reasons. First, parallel workers provide more diverse experience and reduce temporal correlation. Second, short-horizon bootstrapped updates make learning more sample efficient than waiting for full episodes. Third, entropy regularization discourages premature collapse of the continuous-action policy. The resulting 55% success rate indicates that the convolutional encoder learns features sufficient for localizing objects and that the policy head can convert those features into effective grasping actions.

IV. Conclusion

This project implemented and evaluated three reinforcement learning methods spanning tabular control, policy-gradient optimization, and asynchronous vision-based actor-critic learning. In the first part, policy iteration was applied to a finite-state Markov Decision Process and converged efficiently to the optimal policy, illustrating the principles of exact dynamic programming. In the second part, REINFORCE and Advantage Actor-Critic (A2C) were implemented for the `LunarLander-v2` environment, showing how the introduction of a learned value baseline improves training stability and policy performance relative to a pure Monte Carlo policy-gradient method. In the final part, Asynchronous Advantage Actor-Critic (A3C) was used to solve a robotic grasping task from image observations, demonstrating how reinforcement learning can be extended to high-dimensional perception and continuous control problems. Overall, the project highlights that successful reinforcement learning depends not only on the choice of algorithm, but also on the use of appropriate representations, stabilization techniques, and training strategies as task complexity increases.